

<HLS Programming with FPGAs>

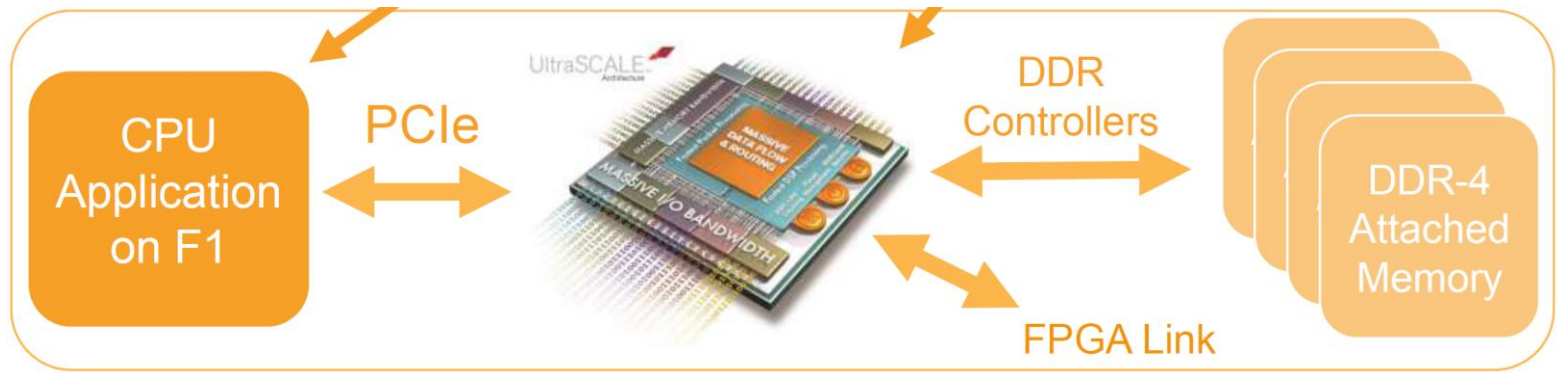
Vitis and Vitis HLS Flows

Choi, Young-kyu

Learning Objectives

- Explain the difference between Vitis and Vitis HLS flows
- Make Vitis project from Vitis HLS project (using FIR filter)

Vitis (on-board) Flow

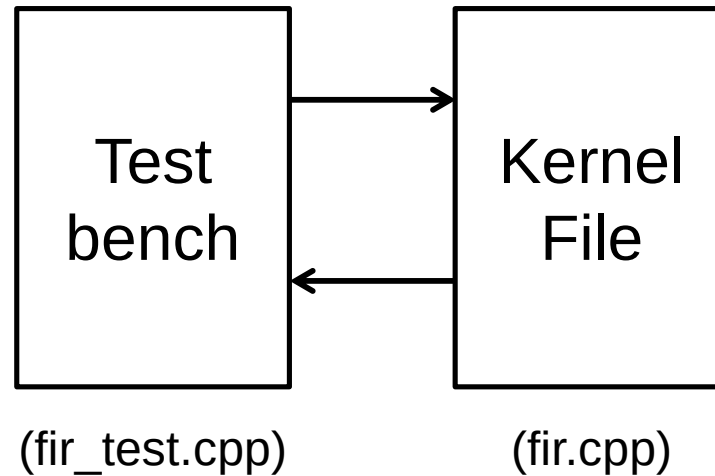


Testbench file
(host.cpp)

Kernel file
(vadd.cpp)

- Testbench file should:
 - Describe CPU behavior
 - Manage PCIe transfer / device memory
 - Start device computation
 - Check correctness of output data given input data

Vitis HLS Flow



- Testbench file should:
 - Check correctness of output data given input data
- Testbench file may (or may not) include
 - CPU / another kernel
 - External memory (DRAM) / BRAM / FIFO / AXI interface

Execution (Simulation) Modes in Vitis / Vitis HLS

	Vitis HLS	Vitis
Algorithm Test	C simulation  C Simulation	SW emulation ** (make run TARGET=sw_emu)
RTL Test	Co-simulation  Co-Simulation	HW emulation ** (make run TARGET=hw_emu)
On-board Execution	-	Build FPGA binary ** (make all TARGET=hw)

(add DEVICE=\$AWS_PLATFORM at end)

** All 3 flows builds host execution file

Command Line Mode for Vitis HLS

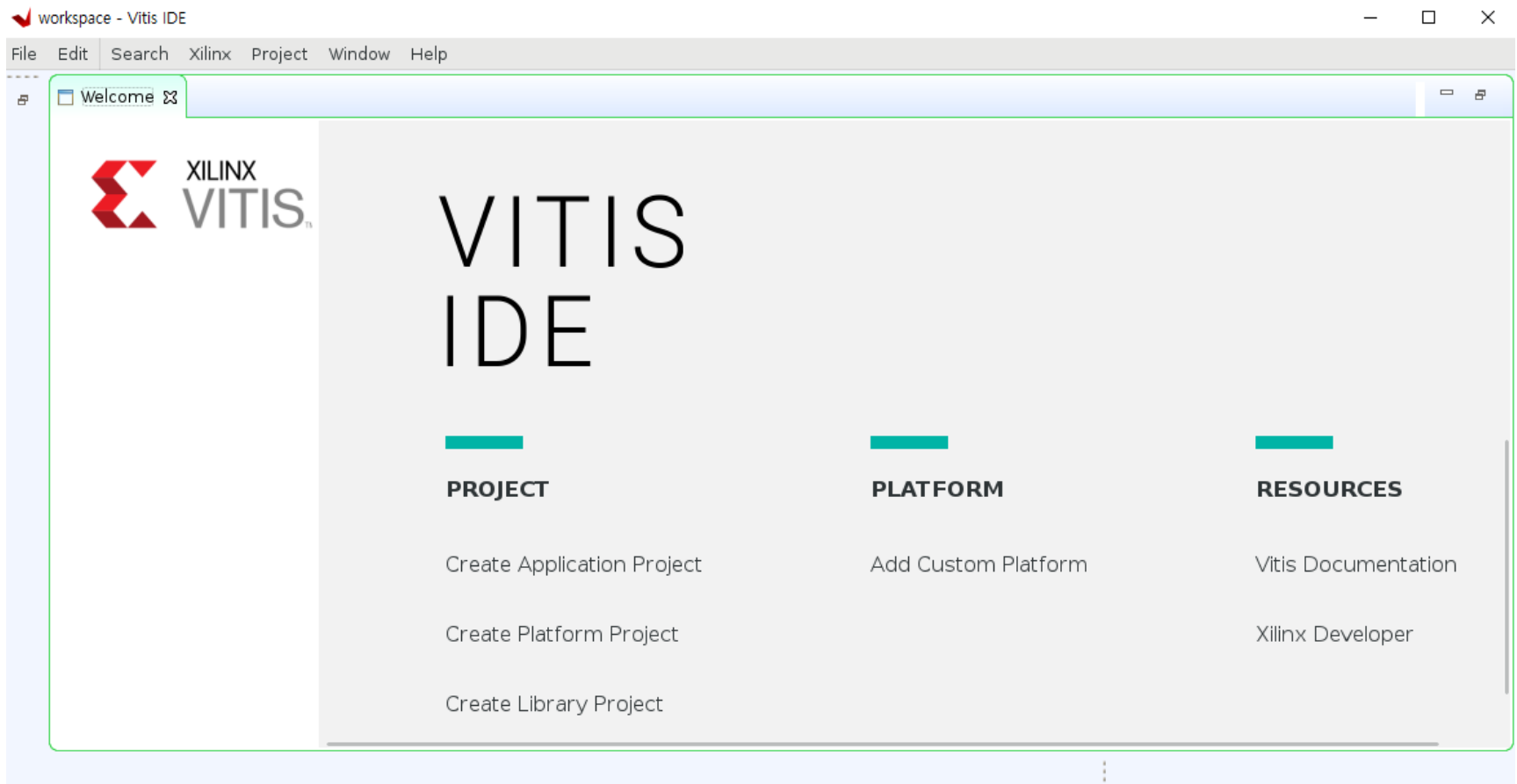
- script.tcl generated in Vitis HLS GUI:

```
1 #####
2 ## This file is generated automatically by Vitis HLS.
3 ## Please DO NOT edit it.
4 ## Copyright 1986-2020 Xilinx, Inc. All Rights Reserved.
5 #####
6 open_project fir
7 set_top fir
8 add_files project1/fir11/fir.h
9 add_files project1/fir11/fir.cpp
10 add_files -tb project1/fir11/fir_test.cpp -cflags "-Wno-unknown-pragmas" -csimflags "-Wno-unknown-pragmas"
11 open_solution "solution1" -flow_target vivado
12 set_part {xcvu9p-flga2104-1-e}
13 create_clock -period 4 -name default
14 set_clock_uncertainty 1.08
15 source "../fir/solution1/directives.tcl"
16 csim_design -clean
17 csynth_design
18 cosim_design -trace_level all
19 export_design -format ip_catalog
```

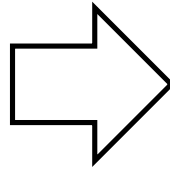
- Read Vitis HLS User Guide (UG1399) for details
- Run “vitis_hls -f script.tcl” in command line

GUI Mode for Vitis

\$ vitis



Typical HLS Development Flow

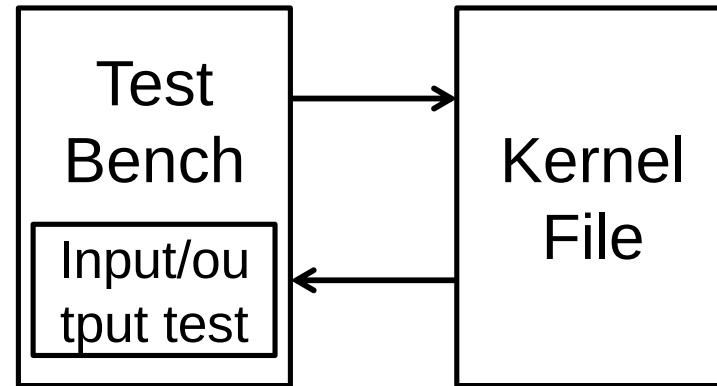


<Step 1>
Initial kernel
development

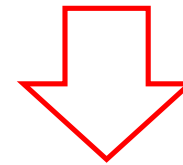
<Step 2>
Add description for
on-board execution

Making Vitis Project from Vitis HLS Project

Vitis HLS view:

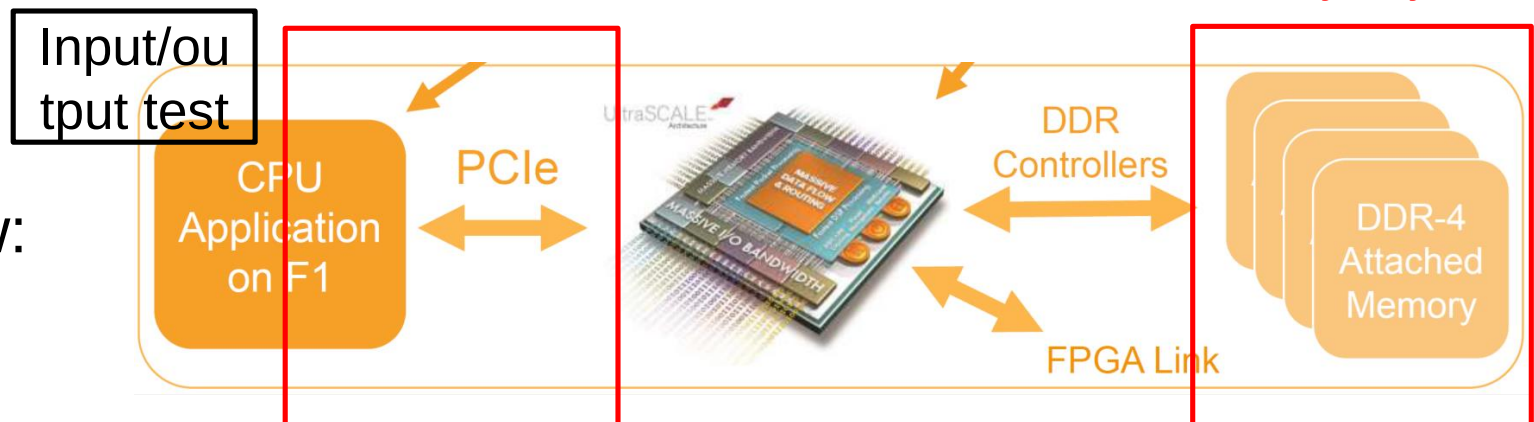


Make PCIE transfer
Invoke kernel



Manage
memory objects

Vitis view:



- Make a Vitis project folder for FIR:

```
cd $VITIS_DIR/examples/xilinx/  
cp -rf hello_world fir_onboard  
cd fir_onboard
```

- Edit Makefile:

```
vim Makefile
```

- Edit directory name:

```
MK_PATH := $(abspath $(lastword $(MAKEFILE_LIST)))  
COMMON_REPO ?= $(shell bash -c 'export MK_PATH=$(MK_PATH); echo $$ {MK_PATH%fir_onboard/*} )'  
PWD = $(shell readlink -f .)  
ABS_COMMON_REPO = $(shell readlink -f $(COMMON_REPO))
```

- Edit host file:
 vim src/host.cpp
 - Creates test data:

```
std::vector<int, aligned_allocator<int>> source_in1(DATA_SIZE);  
std::vector<int, aligned_allocator<int>> source_in2(DATA_SIZE);  
std::vector<int, aligned_allocator<int>> source_hw_results(DATA_SIZE);  
std::vector<int, aligned_allocator<int>> source_sw_results(DATA_SIZE);  
  
// Create the test data  
std::generate(source_in1.begin(), source_in1.end(), std::rand);  
std::generate(source_in2.begin(), source_in2.end(), std::rand);  
for (int i = 0; i < DATA_SIZE; i++) {  
    source_sw_results[i] = source_in1[i] + source_in2[i];  
    source_hw_results[i] = 0;  
}
```

```
std::vector<data_t, aligned_allocator<data_t>> signal(SAMPLES);  
std::vector<data_t, aligned_allocator<data_t>> output(SAMPLES);  
  
FILE* fp, * fin;  
fin = fopen("input.dat", "r");  
fp = fopen("out.dat", "w");  
  
int i;  
for (i = 0; i < SAMPLES; i++) {  
    fscanf(fin, "%d", &signal[i]);  
}  
fclose(fin);
```

- Create memory objects & set argument

```
    OCL_CHECK(err, cl::Buffer buffer_in1(
        context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY,
        vector_size_bytes, source_in1.data(), &err));
    OCL_CHECK(err, cl::Buffer buffer_in2(
        context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY,
        vector_size_bytes, source_in2.data(), &err));
    OCL_CHECK(err, cl::Buffer buffer_output(
        context, CL_MEM_USE_HOST_PTR | CL_MEM_WRITE_ONLY,
        vector_size_bytes, source_hw_results.data(), &err));

    int size = DATA_SIZE;
    OCL_CHECK(err, err = krnl_vector_add.setArg(0, buffer_in1));
    OCL_CHECK(err, err = krnl_vector_add.setArg(1, buffer_in2));
    OCL_CHECK(err, err = krnl_vector_add.setArg(2, buffer_output));
    OCL_CHECK(err, err = krnl_vector_add.setArg(3, size));
```

```
    OCL_CHECK(err, cl::Buffer buffer_x(context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY,
sizeof(data_t)*SAMPLES, signal.data(), &err));
    OCL_CHECK(err, cl::Buffer buffer_y(context, CL_MEM_USE_HOST_PTR | CL_MEM_WRITE_ONLY,
sizeof(data_t)*SAMPLES, output.data(), &err));

    OCL_CHECK(err, err = krnl_vector_add.setArg(0, buffer_y));
    OCL_CHECK(err, err = krnl_vector_add.setArg(1, buffer_x));
```

- Make **PCIE transfer** & **launch kernel**
- (optional) Add **kernel execution time** measurement code

```

OCL_CHECK(err, err = q.enqueueMigrateMemObjects({buffer_x}, 0));
q.finish();

auto start = std::chrono::steady_clock::now();
OCL_CHECK(err, err = q.enqueueTask(krn1_vector_add));
q.finish();
auto end = std::chrono::steady_clock::now();

OCL_CHECK(err, err = q.enqueueMigrateMemObjects({buffer_y},
                                                CL_MIGRATE_MEM_OBJECT_HOST));
q.finish();
double exec_time = std::chrono::duration_cast<std::chrono::nanoseconds>
(end - start).count();
printf("FPGA kernel exec time is %f s\n", exec_time*1e-9);

```

– Output test routine

```
for (i=0;i<SAMPLES;i++){
    fprintf(fp,"%d\n",output[i]);
    printf("%i %d %d\n",i,signal[i],output[i]);
}
fclose(fp);

printf ("Comparing against output data \n");
if (system("diff -w out.dat out.gold.dat")) {
    fprintf(stdout, "*****\n");
    fprintf(stdout, "FAIL: Output DOES NOT match the golden output\n");
    fprintf(stdout, "*****\n");
    return EXIT_FAILURE;
} else {
    fprintf(stdout, "*****\n");
    fprintf(stdout, "PASS: The output matches the golden output!\n");
    fprintf(stdout, "*****\n");
    return EXIT_SUCCESS;
}
```

- Edit kernel file:
vim src/vadd.cpp

```
#include "fir.h"

typedef int coef_t;
typedef int data_t;
typedef int acc_t;

extern "C" { 2. Put extern "C"{} on top function

void vadd( data_t* y, data_t* x ){
#pragma HLS INTERFACE m_axi port = y bundle = gmem0 depth=600
#pragma HLS INTERFACE m_axi port = x bundle = gmem1 depth=600

    coef_t c[N] = {53, 0, -91, 0, 313, 500, 313, 0, -91, 0, 53};

1. Match kernel name
3. Use pointer type and define AXI interface

}

}
```

<Exercise for Fir 2 lecture>

(optional) Define size
of allocated memory
for Vitis HLS co-sim

- View Makefile (optional):

```
include $(ABS_COMMON_REPO)/common/includes/opencv/opencv.mk
CXXFLAGS += $(opencv_CXXFLAGS) -Wall -O0 -g -std=c++11
LDFLAGS += $(opencv_LDFLAGS)

##### Setting up Host Variables #####
#Include Required Host Source Files
CXXFLAGS += -I$(ABS_COMMON_REPO)/common/includes/xcl2
HOST_SRCS += $(ABS_COMMON_REPO)/common/includes/xcl2/xcl2.cpp ./src/host.cpp
# Host compiler global settings
CXXFLAGS += -fmessage-length=0
LDFLAGS += -lrt -lstdc++
```

```
EXECUTABLE = ./hello_world
CMD_ARGS = $(BUILD_DIR)/vadd.xclbin
EMCONFIG_DIR = $(TEMP_DIR)
EMU_DIR = $(SDCARD)/data/emulation

##### Declaring Binary Containers #####
#
BINARY_CONTAINERS += $(BUILD_DIR)/vadd.xclbin
BINARY_CONTAINER_vadd_OBJS += $(TEMP_DIR)/vadd.xo
```

```
$(TEMP_DIR)/vadd.xo: src/vadd.cpp
    mkdir -p $(TEMP_DIR)
    $(VPP) -c -k vadd $(CLFLAGS) --temp_dir $(TEMP_DIR) -I'$(<D)' -o'${@}' '$<'
```


- Copy fir.h and *.dat files into **fir_onboard** directory
- Run simulation
 - make run TARGET=sw_emu DEVICE=\$AWS_PLATFORM**
 - make run TARGET=hw_emu DEVICE=\$AWS_PLATFORM**
- Make bitstream
 - make all TARGET=hw DEVICE=\$AWS_PLATFORM**
- Make AFI and run on F1

Saving Disk Space

- You can delete all the intermediate files in your project folder, except your original source files, host file (hello_world), and AFI (vadd.awsyclbin)

```
description.json  details.rst  Makefile  qor.json  README.rst  src  utils.mk  xrt.ini
```

- Please don't forget to erase .run, .ipcache, build_dir*, and _x* folders

Summary

- Vitis flow: needs to set on-board execution environment
- Vitis HLS flow: higher flexibility in testing kernels
- Made Vitis FIR project from Vitis HLS FIR project